

CAPSTONE PROJECT

Bridging the Clinical Context Gap

An LLM-Powered Multi-Agent System for Synthesizing Electronic Health Records,
Remote Patient Monitoring, and Anamnesis Data

COP-3442: Prompt Engineering

May 13, 2026

Table of Contents

Contents

Table of Contents	2
1. Executive Summary	4
2. Problem Statement: The Clinical Context Gap	5
2.1 Background.....	5
2.1.1 Electronic Health Records (EHR)	5
2.1.2 Remote Patient Monitoring (RPM)	6
2.1.3 Anamnesis	6
2.2 The Core Problem	6
2.3 Why This Problem Fits Prompt Engineering	6
3. Project Objective and Scope	7
3.1 Primary Objective	7
3.2 Scope Boundaries	7
3.3 Target Users (Simulated)	8
4. Proposed Solution Architecture	9
4.1 System Overview	9
4.2 Data Flow	10
5. Module-to-Capstone Mapping.....	11
6. Detailed System Components.....	13
6.1 Alert Triage Agent.....	13
6.2 EHR Retrieval Agent	14
6.3 Anamnesis Agent.....	14
6.4 Synthesis Agent	14
6.5 Orchestrator	15
7. Data Model and Simulated Dataset	16
7.1 Simulated Data Components.....	16
7.2 Scenario Design.....	17
8. Prompt Engineering Specifications	18
8.1 Prompt Documentation Requirements.....	18
8.2 Prompt Design Principles to Demonstrate	19
9. Evaluation Framework.....	20
9.1 Agent-Level Metrics	20

9.2 End-to-End System Metrics	21
9.3 Prompt Engineering Quality Assessment	21
10. Project Milestones and Timeline	22
11. Deliverables Checklist	24
11.1 Core Deliverables.....	24
12. Ethical Considerations and Limitations.....	25
12.1 Critical Disclaimers.....	25
12.2 Ethical Design Principles	25
12.3 Known Limitations	26
13. References	27
13.1 Clinical Context Gap and Health Informatics.....	27
13.2 Remote Patient Monitoring.....	27
13.3 Anamnesis and Patient-Reported Data	28
13.4 LLM Applications in Healthcare.....	28
13.5 Prompt Engineering and Multi-Agent Systems.....	28

1. Executive Summary

This capstone project challenges you to design, build, prompt-engineer, and evaluate a context-aware, multi-agent clinical decision-support prototype that bridges the three most fragmented pillars of patient data: Electronic Health Records (EHR), Remote Patient Monitoring (RPM), and Anamnesis (structured patient history intake). The project is titled **ClinicalBridge**, reflecting its central mission of connecting data sources that currently exist in isolation from one another. You will apply every skill acquired across the eight modules of the Prompt Engineering course, progressing from foundational LLM concepts through advanced multi-agent orchestration. The problem domain — healthcare — was chosen deliberately because it demands precision, safety awareness, and structured reasoning, all of which are hallmarks of excellent prompt engineering. By the end of this capstone, you will have demonstrated that prompt engineering is not merely about writing better instructions for a chatbot, but about architecting intelligent systems that synthesize complex, heterogeneous information into actionable knowledge.

The project synthesizes every module in the Prompt Engineering course into a single, cohesive healthcare application that addresses a real and well-documented challenge in clinical informatics. You will not build a production clinical system; instead, you will create a functional proof-of-concept using simulated clinical data that demonstrates how carefully engineered prompts, retrieval-augmented generation, agent memory, and multi-agent coordination can transform disconnected health data into actionable clinical context. The simulated environment allows you to focus entirely on prompt engineering quality without the regulatory and ethical complexities of working with real patient records. Each agent in the system — from alert triage to clinical synthesis — requires a distinct prompt engineering approach, giving you the opportunity to demonstrate versatility and depth. The progressive complexity of the project mirrors the course structure itself, ensuring that earlier modules provide the foundation upon which later, more advanced techniques are built.

Your final deliverable is a working prototype accompanied by a comprehensive prompt engineering portfolio that documents every design decision, prompt iteration, evaluation result, and lesson learned. This portfolio is as important as the code itself, because it demonstrates your ability to think critically about prompt design, learn from failures, and iterate toward measurable improvement. The evaluation framework built into this capstone requires you to define success metrics before you begin building, test rigorously against clinical scenarios, and reflect on what worked and what did not. You will also be assessed on the quality of your safety guardrails, your handling of uncertainty, and your ability to prevent hallucination in a domain where factual accuracy is paramount. Taken together, the prototype and portfolio represent a complete demonstration of professional-grade prompt engineering applied to a meaningful, high-stakes problem.

2. Problem Statement: The Clinical Context Gap

The healthcare industry is experiencing a data paradox that has profound implications for patient safety, clinical efficiency, and the quality of care delivered worldwide. Despite billions of dollars invested in health information technology over the past two decades, clinicians frequently find themselves making critical decisions without the meaningful context they need. The data they require does exist — scattered across electronic health records, remote monitoring devices, and patient intake forms — but it is fragmented, unstructured, and locked in systems that were never designed to communicate with each other. This fragmentation is not a theoretical concern; it contributes directly to diagnostic errors, delayed treatments, and clinician burnout. Understanding this problem in depth is essential for you, because the entire **ClinicalBridge** system you will build is designed as a response to these specific, well-documented failures in clinical information flow.

Three technologies sit at the center of this paradox, each capturing a different temporal dimension of the patient's health story. Electronic Health Records document the past — the accumulation of diagnoses, treatments, and clinical observations over months and years. Remote Patient Monitoring captures the present — a continuous stream of physiological measurements generated in real time by wearable sensors and connected devices. Anamnesis captures the patient's own voice — their subjective experience of symptoms, their account of what they have been doing at home, their family history, and the medications they take (as opposed to what was prescribed). When these three streams are synthesized, they provide a rich, three-dimensional picture of the patient's current health status in the context of their history and self-reported experience. When they remain disconnected, as they are today, clinicians are forced to assemble this picture manually under pressure, or worse, to make decisions with only one or two dimensions visible.

This section presents each of the three data sources in detail, examines the specific ways in which they fail to integrate, and articulates the clinical consequences of that failure. Read this section not merely as background information, but as the requirements specification for the system you will build. Every design decision in **ClinicalBridge** — from the choice of agent architecture to the structure of individual prompts — should be traced back to a specific aspect of the problem described here. The clinical scenarios presented later in this document are derived directly from the failure modes identified in this analysis. By grounding your capstone in a real, evidence-based problem, you will practice the discipline of building LLM applications that solve genuine needs rather than inventing solutions in search of a problem.

2.1 Background

Modern healthcare has generated a paradox: clinicians have more patient data than ever, yet they frequently make decisions with less meaningful context than they need. Three technologies sit at the center of this paradox, and none of them communicate with each other properly.

2.1.1 Electronic Health Records (EHR)

EHRs capture the patient's past — diagnoses, medications, lab results, and visit notes. However, patient history and problem lists within EHRs can be incomplete and contain frequent errors. For a diagnosis as common as hypertension, studies have found that as many as 30% of identified patients were missing the associated structured diagnostic code. The records exist, but they are unstructured, scattered across incompatible systems, and more than 15 years after interoperability was established as a national goal through the HITECH Act, practitioners still routinely rely on digital faxing to exchange crucial patient information.

2.1.2 Remote Patient Monitoring (RPM)

RPM captures the present — continuous streams of vitals, glucose levels, activity patterns, sleep metrics, and cardiac rhythms from wearables and connected devices. However, continuous data monitoring poses significant challenges for clinicians, including alert fatigue and the burden of reviewing data around the clock, alongside difficulties integrating third-party RPM data seamlessly into existing EHR systems. Research indicates that only 5–13% of ICU alarms are clinically actionable, and as few as 2% of monitored patients can be responsible for generating 77% of all false alarms — a signal-to-noise crisis that causes clinicians to tune out the very alerts that matter most.

2.1.3 Anamnesis

Anamnesis — the structured patient history collected at intake — captures the patient’s own account: symptoms, their onset and progression, lifestyle factors, family history, and medications taken at home. This is the interpretive key that gives context to everything else. Yet healthcare providers spend excessive time on documentation because intake data is often unstructured, which negatively impacts clinical workflows and can create communication barriers during consultations. In some advanced health systems, data collected as part of a medical history is not yet explicitly stored in the EHR, illustrating how anamnesis remains structurally disconnected from the clinical record.

2.2 The Core Problem

When a clinician receives an RPM alert, they cannot rapidly synthesize it against the patient’s longitudinal EHR history and the patient’s own reported context. They are forced to act on incomplete pictures, dismiss alerts out of caution, or spend valuable minutes hunting for context across disconnected systems.

This is not merely an inconvenience. Patient safety concerns when implementing RPM include the risk of clinical misdiagnosis or failure to identify when patients need immediate attention. Clinicians are often required to synthesize combined information from traditional EHRs — repositories of largely episodic data captured in clinical settings — with rapidly growing RPM data consisting of continuous, real-world data generated outside clinical settings. The result is a significant challenge of information overload and a lack of efficient access to data, leading to a critical shortage of actionable clinical insights.

2.3 Why This Problem Fits Prompt Engineering

The Clinical Context Gap is fundamentally a language and reasoning problem. The raw data already exists; what is missing is an intelligent intermediary that can retrieve the right information from each source, interpret it in combination, and present a coherent clinical narrative to the clinician. Large Language Models, when guided by carefully engineered prompts and orchestrated as multi-agent systems, are uniquely positioned to serve as this intermediary — making this problem an ideal capstone for your Prompt Engineering course.

3. Project Objective and Scope

This chapter defines what your **ClinicalBridge** capstone project aims to achieve and, equally importantly, what it deliberately excludes. Establishing clear objectives and scope boundaries at the outset is a critical discipline in any LLM application project, because the open-ended nature of language models makes scope creep an ever-present risk. You must internalize the distinction between a focused proof-of-concept that demonstrates prompt engineering mastery and a production system that would require regulatory approval, clinical validation, and enterprise-grade infrastructure. The objective defined here is ambitious enough to require the full range of skills you have developed across the course, yet constrained enough to be achievable within the capstone timeline. Every subsequent design decision in this document should be evaluated against whether it serves this stated objective.

The primary objective centers on producing a system that can transform a raw RPM alert into a structured, context-rich Clinical Context Brief within a timeframe that is clinically useful. This means your system must not only retrieve relevant data from multiple sources but also reason about how those data points relate to each other and to the triggering alert. The quality of the output is measured not by how much data it surfaces, but by how effectively it reduces the clinician's cognitive load and time-to-decision. Think of the Clinical Context Brief as the “product” of your prompt engineering work — the artifact that a clinician would actually read and act upon. Designing for this end user requires empathy, clarity, and an understanding that in clinical settings, every unnecessary sentence is a cost.

The scope boundaries defined in this chapter protect you from two common failure modes in capstone projects: trying to build too much and losing depth or building the wrong thing and discovering it too late. The in-scope items are directly aligned with the course modules and the skills you have been developing throughout the term. The out-of-scope items are not lesser priorities; they are simply outside the domain of a prompt engineering course and would require expertise in regulatory compliance, clinical informatics infrastructure, or frontend engineering. If you wish to explore any out-of-scope item as a bonus deliverable, you are encouraged to do so, provided it does not compromise the quality of your core prompt engineering work. The target users described here are fictional, but they are based on real clinical roles and workflows to ensure your system design remains grounded.

3.1 Primary Objective

Design and implement a multi-agent, LLM-powered prototype system called **ClinicalBridge** that, given a simulated RPM alert for a patient, automatically retrieves and synthesizes relevant EHR history and anamnesis data to produce a structured, context-rich clinical summary that a clinician can review in under 60 seconds.

3.2 Scope Boundaries

In Scope	Out of Scope
• Simulated EHR, RPM, and anamnesis datasets • Multi-agent architecture with specialized agents • Prompt engineering for each agent role • RAG pipeline for EHR context retrieval	• Real patient data or HIPAA-regulated systems • Production-grade clinical deployment • FDA or regulatory compliance testing • Real-time wearable device integration

• Memory and tool integration for agents • Evaluation framework with defined metrics • Prompt iteration portfolio with version history	• Custom LLM fine-tuning or model training • Full EHR interoperability (HL7/FHIR at scale) • User interface or frontend development
--	---

3.3 Target Users (Simulated)

Your prototype is designed for a simulated audience of primary care physicians and remote monitoring nurses who receive RPM alerts for patients with chronic conditions such as hypertension, diabetes, and heart failure. The system assumes that the clinician needs a synthesized context brief, not raw data.

4. Proposed Solution Architecture

The architecture of **ClinicalBridge** is designed around the principle that complex clinical reasoning is best decomposed into specialized subtasks, each handled by a purpose-built agent with its own prompt, memory, and tools. This mirrors how clinical teams operate in practice: a triage nurse evaluates urgency, a specialist reviews the medical record, the patient provides their own history, and a senior clinician synthesizes everything into a decision. By assigning each of these cognitive roles to a distinct LLM agent, you can engineer focused, testable prompts rather than attempting to build a single monolithic prompt that handles everything. This decomposition also makes your system more transparent, because each agent's contribution to the final output can be traced and evaluated independently. The architecture represents the practical application of concepts from Modules 7 and 8 of the course, where you learned to build autonomous agents and coordinate them into multi-agent systems.

ClinicalBridge is organized as a four-agent system coordinated by a central orchestrator, for a total of five distinct components. Each agent is a prompt-engineered LLM instance with a specialized role, dedicated memory, and access to specific tools that constrain and empower its behavior. The orchestrator does not perform clinical reasoning itself; its role is purely coordinative, managing the flow of information between agents, handling errors and timeouts, and enforcing safety guardrails that operate at the system level rather than the agent level. This separation of concerns is a deliberate architectural choice that you should understand deeply. It means that improving the system's triage accuracy, for example, requires modifying only the Triage Agent's prompt, not the entire system — a property that makes iterative prompt engineering practical and measurable.

The data flow through the system follows a linear-then-convergent pattern that you will implement using LangChain workflows and agent orchestration patterns. An alert enters the system and is processed sequentially by the Triage Agent, which then triggers parallel retrieval from the EHR and Anamnesis agents, before all results converge at the Synthesis Agent for final output generation. This pattern was chosen because it balances latency (parallel retrieval is faster than sequential) with correctness (the triage step must complete before retrieval begins, because it determines what to retrieve). Pay careful attention to how information is passed between agents, because the quality of inter-agent communication is often the difference between a coherent final output and a disjointed one. The architecture diagram and data flow described below should serve as the blueprint that you implement progressively across the milestone timeline.

4.1 System Overview

Agent	Role	Data Source	Key Module(s)
Alert Triage Agent	Receives RPM alerts, classifies urgency, identifies the patient, and formulates context queries	RPM data stream	M2, M3, M4
EHR Retrieval Agent	Searches the patient's longitudinal EHR using RAG to extract relevant diagnoses, medications, labs, and visit notes	EHR vector store	M6, M7
Anamnesis Agent	Retrieves and interprets the patient's self-reported history, including recent symptom logs, lifestyle factors, and medication adherence	Anamnesis records	M3, M4, M7

Synthesis Agent	Combines outputs from all three agents into a coherent, prioritized clinical context brief with actionable recommendations	Agent outputs	M4, M6, M8
Orchestrator	Routes the workflow, manages inter-agent communication, handles errors, and enforces clinical safety guardrails	All agents	M7, M8

4.2 Data Flow

1. **Alert Ingestion:** An RPM alert (e.g., sustained elevated blood pressure) is received by the Alert Triage Agent.
2. **Triage and Query Formulation:** The Triage Agent classifies the alert severity, identifies the patient, and generates structured queries for the EHR and Anamnesis agents.
3. **Parallel Retrieval:** The Orchestrator dispatches queries to the EHR Retrieval Agent and the Anamnesis Agent in parallel.
4. **Context Extraction:** Each retrieval agent searches its respective data store, extracts relevant information, and returns structured findings.
5. **Synthesis:** The Synthesis Agent receives all findings, correlates them against the original alert, and produces a Clinical Context Brief.
6. **Output:** The final brief is returned with a confidence score, cited sources, and flagged uncertainties.

5. Module-to-Capstone Mapping

One of the defining characteristics of a well-designed capstone project is that it requires you to integrate and apply knowledge from across the entire curriculum, rather than drawing on just one or two modules. This chapter demonstrates that every module in the Prompt Engineering course has a direct, substantive application within the **ClinicalBridge** project. The mapping is not superficial or forced; each module addresses a genuine need that arises naturally from the problem domain and the chosen architecture. Use this mapping as a self-assessment tool throughout your project: if you find that your implementation does not meaningfully exercise a particular module's concepts, that is a signal to revisit your design. The progression from Module 1 through Module 8 also mirrors the natural build order of the project, from foundational understanding through to full system integration.

The table presented in this chapter should be read as a contract between you and the course. For each module, the capstone application column describes how the module's concepts are used within **ClinicalBridge**, and the deliverable artifact column specifies what tangible evidence you must produce to demonstrate that application. These are not vague expectations; they are specific, assessable outputs that will be reviewed as part of your final evaluation. If you complete all deliverable artifacts across all eight modules, you will have produced a portfolio that demonstrates both breadth and depth in prompt engineering. The mapping also helps you plan your work, because it makes explicit which module's skills are needed at each stage of the project timeline.

It is worth emphasizing that the later modules (Modules 6 through 8) build directly on the earlier ones, and that weaknesses in foundational prompt design will cascade into the advanced components. A poorly designed system prompt from Module 3, for example, will undermine the effectiveness of the RAG pipeline you build in Module 6 and the multi-agent coordination you implement in Module 8. Treat the early milestones with the same rigor as the later ones, and resist the temptation to rush through prompt design in order to reach the more technically exciting agent and multi-agent work. The iterative nature of prompt engineering means that you will return to earlier prompts repeatedly as you discover issues in later stages — this is not a sign of failure, but a sign of mature engineering practice.

Course Module	Capstone Application	Deliverable Artifact
M1: Intro to LLMs & Prompt Engineering	Understanding LLM capabilities and limitations for clinical text; selecting appropriate models; establishing prompt engineering fundamentals	Model selection rationale document; baseline prompt experiments
M2: Designing LLM Applications	Designing the overall ClinicalBridge application architecture; defining user personas (clinicians); specifying input/output contracts for each agent	Application design document; agent interface specifications
M3: Prompt Content & Assembling the Prompt	Crafting system prompts, few-shot examples, and structured output templates for each of the four agents; designing the Clinical Context Brief template	Prompt library with version history; output schema definitions
M4: Conversational Agency & LLM Workflows	Implementing the conversational flow between agents; designing the Anamnesis Agent's patient interview	Conversation flow diagrams; chain-of-thought prompt designs

	simulation; building the Synthesis Agent's reasoning chain	
M5: Testing LLM Applications	Building the evaluation framework; designing test cases with known-correct clinical scenarios; measuring accuracy, relevance, latency, and hallucination rates	Test suite; evaluation results dashboard; regression test set
M6: Advanced Techniques with LangChain	Implementing RAG for EHR retrieval; building LangChain chains and pipelines; integrating structured output parsing; managing prompt templates programmatically	LangChain pipeline code; RAG configuration; retrieval evaluation
M7: Autonomous Agents with Memory & Tools	Equipping each agent with appropriate memory (conversation, summary, entity); integrating tools (vector store search, data parsing, alert classification); enabling autonomous operation within defined boundaries	Agent implementation code; memory architecture documentation; tool integration tests
M8: Multi-Agent System	Orchestrating all agents into a coordinated system; implementing inter-agent communication protocols; building error handling and fallback strategies; enforcing clinical safety guardrails	Complete multi-agent system; orchestration logic; end-to-end demonstration

6. Detailed System Components

This chapter provides detailed specifications for each of the five components that comprise the **ClinicalBridge** system: the four specialized agents and the central orchestrator. Each component is described in terms of its purpose, its inputs and outputs, and the specific prompt engineering challenges it presents. These specifications serve as the functional requirements you must implement, and they are intentionally detailed enough to guide your implementation while leaving room for you to make your own design decisions about prompt structure, few-shot example selection, and chain-of-thought strategies. Read each component specification as a design brief, not as a prescriptive recipe.

The prompt engineering focus areas listed for each agent are particularly important, because they represent the core intellectual contribution of your capstone. Building a system that calls an LLM API is straightforward; building a system that calls an LLM API with precisely engineered prompts that produce reliable, safe, and clinically appropriate outputs is the challenge that distinguishes this project from a simple integration exercise. Each agent's prompt must be developed through multiple iterations, tested against clinical scenarios, and documented with the kind of rigor described in Chapter 8. Expect to spend more time designing and refining prompts than writing the code that calls the API. This ratio — more prompt engineering than software engineering — is intentional and reflects the real-world distribution of effort in LLM application development.

The components are presented in the order they appear in the system's data flow, from alert ingestion through to final synthesis. However, you need not implement them in this order; the milestone timeline in Chapter 10 recommends a different build sequence that accounts for dependencies and allows for early testing. Each component's specification includes references to the course modules that are most relevant, helping you connect your implementation work back to the theoretical foundations you have studied. The orchestrator is described last because it depends on all other components, but you should design it early, because decisions about orchestration patterns (sequential, parallel, conditional) will influence how your individual agents are structured.

6.1 Alert Triage Agent

Purpose: Receive incoming RPM alerts, classify their clinical urgency, extract the patient identifier, and formulate precise retrieval queries for downstream agents.

Input: A structured RPM alert object containing patient ID, timestamp, device type, measured values, device-level alert category, and the patient's baseline thresholds.

Output: A triage decision object containing urgency classification (Critical, Urgent, Routine, Informational), a natural-language clinical question summarizing the concern, and structured query parameters for the EHR and Anamnesis agents.

Prompt Engineering Focus:

- System prompt with clinical triage guidelines and urgency definitions
- Few-shot examples mapping RPM alert patterns to triage levels
- Chain-of-thought reasoning to justify urgency classification
- Output structured as JSON with enforced schema validation

6.2 EHR Retrieval Agent

Purpose: Search the patient's longitudinal electronic health record using retrieval-augmented generation to extract diagnoses, medications, laboratory results, and visit notes relevant to the current alert.

Input: Patient ID, structured query parameters from the Triage Agent, and the clinical question.

Output: A structured EHR context object containing relevant problem list entries, active medications, recent lab results with trends, pertinent visit note excerpts, and metadata such as retrieval confidence scores and source document references.

Prompt Engineering Focus:

- RAG pipeline design: chunking strategy for clinical documents, embedding model selection, retrieval parameters
- System prompt that instructs the LLM to act as a clinical data analyst, not a diagnostician
- Handling of incomplete or contradictory EHR entries
- Explicit instructions to flag missing data rather than hallucinate clinical facts

6.3 Anamnesis Agent

Purpose: Retrieve and interpret the patient's self-reported history, including recent symptom logs, lifestyle and dietary factors, medication adherence records, and family history relevant to the current alert.

Input: Patient ID, the clinical question from the Triage Agent, and any specific anamnesis categories flagged as relevant.

Output: A structured anamnesis summary containing recent patient-reported symptoms and timeline, medication adherence status, relevant lifestyle factors, family history highlights, and patient-expressed concerns or questions.

Prompt Engineering Focus:

- Conversational prompt design for simulating patient intake interviews
- Interpretation prompts that translate colloquial patient language into clinical terminology
- Memory integration to track patient-reported information over multiple interactions
- Sensitivity guardrails for handling mental health and substance use disclosures

6.4 Synthesis Agent

Purpose: Combine the outputs of the Triage, EHR Retrieval, and Anamnesis agents into a single, coherent Clinical Context Brief that presents the clinician with an actionable understanding of the situation.

Input: The triage decision, EHR context object, anamnesis summary, and the original RPM alert data.

Output: A Clinical Context Brief (CCB) structured as follows:

1. **Alert Summary:** What triggered the alert and its classified urgency.
2. **Patient Snapshot:** Key demographics, active conditions, and current treatment plan.

3. **Contextual Analysis:** How the alert relates to the patient's history, recent trends, and self-reported status.
4. **Risk Assessment:** Potential clinical implications and differential considerations.
5. **Recommended Actions:** Suggested next steps with confidence levels and supporting evidence.
6. **Uncertainties and Gaps:** Explicitly flagged missing data, conflicting information, or areas requiring clinician judgment.

Prompt Engineering Focus:

- Multi-source synthesis prompting with explicit citation requirements
- Clinical reasoning chain-of-thought that mirrors differential diagnosis methodology
- Confidence calibration prompts that encourage the model to express uncertainty appropriately
- Anti-hallucination guardrails: every claim must reference a specific source from the upstream agents

6.5 Orchestrator

Purpose: Coordinate the entire multi-agent workflow, manage inter-agent communication, handle errors and timeouts, enforce clinical safety guardrails, and ensure the final output meets quality standards before delivery.

Key Responsibilities:

- Route the workflow from alert ingestion through synthesis to output delivery
- Manage parallel dispatching of queries to the EHR and Anamnesis agents
- Implement retry logic and fallback strategies when an agent fails or returns low-confidence results
- Enforce safety guardrails: escalate critical alerts that require immediate human attention without waiting for full synthesis
- Maintain a session log of all inter-agent communications for auditability

7. Data Model and Simulated Dataset

The quality of any LLM-powered system is fundamentally constrained by the quality and realism of the data it operates on, and this is especially true in healthcare applications where clinical plausibility is essential for meaningful evaluation. This chapter specifies the simulated dataset that you must create to serve as the foundation for all your **ClinicalBridge** development and testing. The dataset represents a cohort of 10 to 20 fictional patients with chronic conditions, and it must be entirely fabricated — no real patient data of any kind may be used at any stage of this project. Despite being fictional, the data must be clinically plausible, internally consistent, and rich enough to support the five clinical scenarios described below. Invest significant effort in dataset creation, because a poorly designed dataset will make it impossible to distinguish between prompt engineering failures and data quality failures during evaluation.

The simulated dataset has three components corresponding to the three data sources at the heart of the Clinical Context Gap: EHR records capturing the patient’s medical history, RPM data streams capturing continuous physiological measurements, and anamnesis records capturing the patient’s self-reported health information. Each component has a different structure and format, reflecting the real-world heterogeneity that **ClinicalBridge** must navigate. EHR records are document-oriented and will be chunked and embedded in a vector store for retrieval-augmented generation. RPM data is time-series and numeric, requiring the Triage Agent to interpret trends and thresholds. Anamnesis records are narrative and semi-structured, requiring the Anamnesis Agent to extract meaning from conversational language. This diversity of data formats is not an accident; it is a deliberate design feature that forces you to engineer different prompt strategies for different data types.

Beyond the raw data, this chapter also specifies the clinical scenarios that you must design as end-to-end test cases for your system. Each scenario is a complete vignette that includes a triggering RPM alert, the relevant EHR history, the pertinent anamnesis data, and a gold-standard Clinical Context Brief that represents the ideal system output. These scenarios are not arbitrary; they are designed to test specific capabilities and edge cases, such as handling missing data, resolving conflicting information, detecting gradual trends, and distinguishing true alerts from false alarms. You are required to create at least five scenarios, but you are encouraged to create more to strengthen your evaluation. The scenarios also serve as the primary vehicle for prompt iteration, because each failed scenario provides specific, actionable feedback about what your prompts need to handle better.

7.1 Simulated Data Components

Data Source	Contents	Format
EHR Records	Demographics, problem lists (ICD-10 coded), medication lists, lab results with timestamps, visit notes (free text), allergy records	JSON documents chunked and embedded in a vector store (e.g., ChromaDB, FAISS)
RPM Data Streams	Timestamped vital sign readings (blood pressure, heart rate, blood glucose, SpO2, weight), device metadata, alert trigger thresholds, historical baselines	Structured JSON or CSV time-series data
Anamnesis Records	Structured intake forms (chief complaint, history of present illness, review of systems, social history, family history), patient symptom diaries, medication adherence logs	Semi-structured text documents with JSON metadata

7.2 Scenario Design

You must design at least five distinct clinical scenarios that test different aspects of your system. Each scenario should include a triggering RPM alert, relevant EHR history, pertinent anamnesis data, and a gold-standard Clinical Context Brief that you write yourself for evaluation comparison.

Example scenarios:

1. **The Missed Medication:** A hypertensive patient's blood pressure spikes; the anamnesis reveals they stopped taking their ACE inhibitor due to a persistent cough — a known side effect not flagged in the EHR.
2. **The False Alarm:** A diabetic patient's glucose reading triggers an alert, but the anamnesis records a planned dietary change, and the EHR shows a recent medication adjustment — the alert is contextually benign.
3. **The Silent Deterioration:** A heart failure patient's weight increases gradually over two weeks; individually each reading is within threshold, but the trend, combined with the patient's reported increased ankle swelling in the anamnesis, suggests fluid retention.
4. **The Incomplete Record:** A patient transfers from another system; the EHR is sparse, and your system must rely heavily on anamnesis data while clearly flagging the gaps.
5. **The Conflicting Data:** The patient reports full medication adherence in the anamnesis, but lab results in the EHR show sub-therapeutic drug levels — your system must flag the discrepancy without making accusations.

8. Prompt Engineering Specifications

Prompt design is the central intellectual contribution of this capstone, and this chapter defines the standards to which all your prompt work will be held. While the system architecture, data model, and agent code are all necessary components, they are ultimately in service of the prompts that drive each agent's behavior. A well-designed prompt can compensate for a simple architecture; a poorly designed prompt will undermine even the most sophisticated pipeline. Approach prompt engineering with the same rigor and discipline you would bring to writing production code: version control, testing, documentation, and iterative refinement are not optional practices but required ones. This chapter specifies both what you must document and what principles you must demonstrate.

The documentation requirements defined here are designed to make your prompt engineering work visible, assessable, and reproducible. It is tempting to treat prompts as informal, easily modified text strings, but in the context of a clinical decision-support system, every word in a prompt has consequences for the quality and safety of the output. By requiring you to maintain a version history, document your rationale for changes, and analyze failure modes, this specification forces the kind of reflective practice that distinguishes expert prompt engineers from beginners. The iteration log, in particular, is a critical artifact: it demonstrates not just what you built, but how you thought about it, what went wrong, and how you responded. Evaluators will look for evidence of genuine learning and adaptation, not just a polished final prompt.

The prompt design principles listed in this chapter are drawn from best practices in the field and from the course curriculum. They are not abstract guidelines; each principle is directly applicable to the clinical agents in **ClinicalBridge** and should be demonstrable in your prompt portfolio. For example, the principle of chain-of-thought reasoning is not merely a technique you should know about — it is a strategy you should implement in the Synthesis Agent's prompt to ensure that clinical reasoning steps are explicit and auditable. Similarly, anti-hallucination strategies are not a theoretical concern in this project; they are a safety requirement, because a hallucinated medication or lab value in a Clinical Context Brief could, in a real-world deployment, lead to clinical harm. Treat these principles as a checklist and ensure that every prompt in your portfolio demonstrates at least the majority of them.

8.1 Prompt Documentation Requirements

For each agent, you must document:

1. **System Prompt:** The complete system prompt including role definition, behavioral constraints, output format specifications, and safety guardrails.
2. **Few-Shot Examples:** At least three input/output examples that demonstrate the expected behavior, including at least one edge case.
3. **Chain-of-Thought Design:** Where applicable, the reasoning structure the model should follow, with explicit intermediate steps.
4. **Output Schema:** The structured output format (JSON schema, template, or structured text format) with field descriptions and validation rules.
5. **Iteration Log:** A version history of at least three prompt iterations per agent, with before/after comparisons and rationale for changes.
6. **Failure Mode Analysis:** Documented cases where the prompt produced incorrect, hallucinated, or unsafe outputs, and how the prompt was revised to address them.

8.2 Prompt Design Principles to Demonstrate

- Clarity and specificity in role definition and task instructions
- Effective use of delimiters and structured formatting to separate context from instructions
- Positive and negative examples (what to do and what not to do)
- Output format enforcement through schema specification and validation
- Chain-of-thought and step-by-step reasoning for complex clinical analysis
- Temperature and parameter tuning with documented rationale
- Safety guardrails: explicit instructions to avoid diagnosis, flag uncertainty, and defer to clinicians
- Anti-hallucination strategies: source citation requirements, confidence scoring, and explicit “unknown” handling

9. Evaluation Framework

Evaluation is the discipline that separates principled prompt engineering from trial-and-error tinkering, and this chapter defines the comprehensive framework through which all your **ClinicalBridge** work will be assessed. In the context of LLM applications, evaluation is particularly challenging because outputs are often open-ended text rather than discrete, easily verifiable answers. A clinical context brief might be factually correct but poorly organized, or well-organized but missing a critical piece of information, or complete and well-structured but containing a single hallucinated detail that undermines the entire output. Your evaluation framework must therefore be multi-dimensional, capturing different facets of quality at different levels of the system. Build this framework early in your project and use it continuously throughout development, not just as a final assessment.

The framework is structured at three levels that correspond to increasing scope of analysis. Agent-level metrics evaluate each component in isolation, measuring whether it performs its specific function correctly and reliably. End-to-end system metrics evaluate the complete pipeline, measuring whether the orchestrated agents produce a final output that meets clinical and operational standards. Prompt engineering quality metrics evaluate your process and documentation, measuring whether your prompt design work demonstrates the depth, rigor, and reflective practice expected at capstone level. These three levels are complementary, not redundant: a system can achieve good end-to-end results despite a weak individual agent (if other agents compensate), or it can have strong individual agents that produce poor end-to-end results (if orchestration is flawed). You must evaluate at all three levels to gain a complete picture.

The specific metrics, measurement methods, and targets defined in this chapter are recommendations, not rigid requirements. You are encouraged to adapt and extend the framework based on what you discover during development. If a particular failure mode emerges that is not captured by the defined metrics, propose and implement an additional metric to address it. Similarly, if a target proves unrealistic given the limitations of current LLM capabilities, document this finding and propose a revised target with justification. The evaluation framework is itself a deliverable, and it will be assessed for its thoughtfulness, completeness, and practical utility. An honest evaluation that reveals weaknesses is valued far more than a superficial evaluation that hides them.

9.1 Agent-Level Metrics

Agent	Metric	Measurement Method	Target
Triage	Urgency classification accuracy	Compare against gold-standard labels across all test scenarios	≥90%
Triage	Query relevance	Expert review of generated queries for clinical appropriateness	≥4/5 rating
EHR Retrieval	Retrieval precision	Proportion of retrieved chunks relevant to the clinical question	≥80%
EHR Retrieval	Retrieval recall	Proportion of relevant EHR facts successfully retrieved	≥75%

Anamnesis	Information extraction completeness	Percentage of relevant anamnesis fields populated vs. gold standard	≥85%
Anamnesis	Interpretation accuracy	Correct mapping of patient language to clinical concepts	≥80%
Synthesis	Clinical accuracy	Factual correctness of all claims in the CCB against source data	≥90%
Synthesis	Hallucination rate	Percentage of claims not supported by any source agent output	≤5%
Synthesis	Completeness	Coverage of all relevant clinical dimensions in the CCB	≥85%

9.2 End-to-End System Metrics

1. **Scenario Pass Rate:** Percentage of test scenarios where the final CCB correctly identifies the primary clinical concern, relevant context, and appropriate recommendations.
2. **Time-to-Brief:** End-to-end latency from alert ingestion to CCB generation; target is under 30 seconds for non-critical alerts.
3. **Safety Compliance:** Percentage of outputs that correctly include uncertainty flags, avoid making diagnoses, and defer appropriately to clinical judgment.
4. **Source Traceability:** Percentage of claims in the CCB that include traceable references to specific EHR entries, RPM readings, or anamnesis records.

9.3 Prompt Engineering Quality Assessment

- Iteration depth: demonstrated improvement across at least three versions per agent prompt
- Failure analysis quality: thoroughness of documented failure modes and revisions
- Design rationale: clarity of reasoning behind prompt design decisions
- Generalizability: performance consistency across diverse clinical scenarios
- Safety consciousness: evidence of proactive safety considerations in prompt design

10. Project Milestones and Timeline

Effective project management is essential for a capstone of this complexity, and this chapter provides a structured timeline that decomposes the work into eight milestones across a 5 week period. The milestones are deliberately aligned with the course modules, so that you can apply concepts shortly after learning them rather than trying to remember and integrate them all at the end. Each milestone has defined activities and a specific deliverable, creating natural checkpoints where you can assess your progress and course-correct if needed. The timeline assumes that you are working on the capstone concurrently with the later course modules, which means that the early milestones focus on foundation and design work that can proceed before the advanced LangChain and multi-agent content has been covered.

The timeline is designed with intentional front-loading of design and prompt development work, followed by implementation and integration in the middle weeks, and culminating in comprehensive evaluation and documentation in the final weeks. This structure reflects a key lesson in LLM application development: the temptation to start coding immediately and iterate on prompts later is almost always counterproductive. If you invest in thorough prompt design during Weeks 4 and 5, you will find that your implementation work in Weeks 8 and 9 proceeds much more smoothly, because the agent behaviors are well-defined and the expected inputs and outputs are clearly specified. Conversely, if you rush to implementation, you will spend your integration weeks debugging prompt issues rather than refining system-level behavior.

Treat this timeline as a strong recommendation rather than an inflexible mandate. Your individual circumstances may require adjustments, and the nature of prompt engineering is inherently iterative, meaning that some milestones may need to be revisited as later work reveals issues in earlier designs. However, the deliverables for each milestone are firm requirements: each must be completed and submitted as part of your final portfolio, regardless of when they were produced. If you fall behind the recommended schedule, prioritize the current milestone's deliverable rather than trying to catch up on all missed work simultaneously. The final milestone, in Week 12, is deliberately reserved for evaluation, documentation, and reflection, because these activities require a complete system and cannot be meaningfully started earlier.

Week	Milestone	Activities	Deliverable
1	Foundation & Design	Conduct literature review on the clinical context gap; run model selection experiments and baseline prompt tests; design the simulated patient dataset schema and generate initial records; define agent roles, interfaces, and input/output contracts; create the system architecture diagram and data flow specification	Problem analysis report; model comparison document; dataset schema and initial records; application design document; architecture diagram
2	Prompt Engineering & RAG	Develop system prompts, few-shot examples, output schemas, and chain-of-thought designs for all four agents; design the Clinical Context Brief template; begin the prompt iteration cycle; build the RAG pipeline for EHR retrieval including chunking strategy,	Prompt library v1 for all agents; output schema definitions; working RAG pipeline; conversation flow documentation; initial retrieval evaluation

		embedding model selection, and vector store setup; implement conversational flows and the anamnesis interview simulation	
3	Agent Implementation	Implement all four agents using LangChain chains and pipelines; integrate agent memory (conversation, summary, entity) and tools (vector store search, data parsing, alert classification); build agent reasoning chains; design test cases from clinical scenarios; build the evaluation harness; run the first evaluation cycle and document failure modes; begin prompt iteration v2 based on test results	Working agent prototypes with memory and tools; test suite; first evaluation results; failure analysis v1; prompt library v2
4	Multi-Agent Integration	Orchestrate all agents into the coordinated ClinicalBridge system; implement inter-agent communication protocols and the orchestrator's routing logic; build safety guardrails and escalation pathways; implement retry logic and fallback strategies; run end-to-end testing across all five clinical scenarios; iterate on prompts based on integration findings	Complete integrated multi-agent system; orchestration logic; safety guardrail implementation; end-to-end test results across all scenarios; prompt library v3
5	Evaluation & Portfolio	Run the final comprehensive evaluation across all scenarios and all metrics (agent-level, end-to-end, prompt quality); compile the complete prompt engineering portfolio with all iteration logs, rationale, and failure analyses; write the reflective analysis and update the project report with implementation details and results; record the demonstration walkthrough showing at least two scenarios end-to-end; organize all deliverables in the submission package with a clear README	Final evaluation report; complete prompt engineering portfolio; updated project report; demonstration recording; organized submission repository

11. Deliverables Checklist

This chapter provides a comprehensive and definitive list of all artifacts that you must submit as part of your capstone project. The deliverables are organized into core requirements, which you must complete to pass the capstone, and optional bonus deliverables, which provide opportunities for you to demonstrate additional depth or breadth. The distinction between core and bonus is important: make sure that all core deliverables meet the quality standards defined in this document before investing time in bonus work. A complete set of high-quality core deliverables will always be valued more than a partial set of core deliverables supplemented by impressive but tangential bonus items.

Each core deliverable corresponds to a specific aspect of the capstone project and is designed to be assessable against the criteria defined in the evaluation framework in Chapter 9. The project report documents your understanding of the problem and your design decisions. The working prototype demonstrates your technical implementation and system integration. The simulated dataset shows your ability to create realistic test data that challenges the system meaningfully. The prompt engineering portfolio — arguably the most important deliverable in a prompt engineering course — reveals your thinking process, iterative methodology, and ability to learn from failure. The evaluation report presents your honest, rigorous assessment of system performance. The demonstration ties everything together into a compelling narrative.

Organize all deliverables in a single project repository or submission package with clear, logical structure and a top-level README that explains how to navigate the materials. Your code should be runnable, your prompts should be clearly labeled with version numbers, and your evaluation results should include both raw data and summary analysis. Documentation quality matters: deliverables that are technically sound but poorly organized or explained will receive lower assessments than equally sound work that is clearly presented. Remember that your deliverables may be reviewed by evaluators who are not familiar with your specific implementation choices, so every deliverable should be self-contained enough to be understood without requiring a verbal explanation.

11.1 Core Deliverables

1. **Project Report (this document, completed):** The final version of this capstone definition document, updated with your implementation details, results, and reflections.
2. **Working Prototype:** The complete **ClinicalBridge** system with all agents operational, runnable from a single entry point (e.g., a Python script or Jupyter notebook) that accepts an RPM alert and produces a Clinical Context Brief.
3. **Simulated Dataset:** All EHR records, RPM data streams, and anamnesis records for your patient cohort, with documentation of how they were generated.
4. **Prompt Engineering Portfolio:** A comprehensive document containing every prompt version for each agent, with iteration logs, rationale, before/after comparisons, and failure analyses.
5. **Evaluation Report:** Complete evaluation results across all metrics, including scenario-by-scenario analysis, aggregated statistics, and comparison of prompt versions.
6. **Demonstration:** A recorded walkthrough (video or annotated notebook) showing the system processing at least two different clinical scenarios end-to-end.

12. Ethical Considerations and Limitations

Building an LLM-powered system in the healthcare domain carries ethical responsibilities that extend far beyond the technical requirements of the project. Even though **ClinicalBridge** operates exclusively on simulated data and is explicitly framed as an educational prototype, the design decisions you make and the habits you develop during this capstone will shape how you approach LLM applications in sensitive domains throughout your career. This chapter therefore serves a dual purpose: it defines the ethical guardrails that must be present in your capstone submission, and it asks you to engage in the kind of ethical reasoning that is essential for responsible AI development. Do not treat this chapter as a compliance exercise; treat it as an opportunity to develop the critical thinking skills that distinguish responsible technologists from mere capability builders.

The critical disclaimers and ethical design principles defined below are non-negotiable requirements of your capstone project. Every system output must include appropriate uncertainty disclosures. Your system must never present itself as a diagnostic tool or suggest that its outputs can replace clinical judgment. All data must be simulated, and you must not attempt to use real patient records even if you have access to de-identified datasets, because the risks of re-identification and the regulatory complexities of health data are beyond the scope of this course. The human-in-the-loop principle must be architecturally enforced, not merely stated — meaning your system’s design must assume that a clinician will review every output before any action is taken. These requirements exist not because they make the project harder, but because they make the project better: designing for safety and transparency produces more robust and more useful systems.

Equally important is an honest acknowledgment of your system’s limitations, which are substantial. LLM hallucination risk remains non-zero regardless of how many guardrails are implemented, and clinical applications have a lower tolerance for errors than most other domains. Your simulated data, however carefully constructed, cannot capture the full complexity and messiness of real clinical records with their abbreviations, inconsistencies, and missing fields. The multi-agent architecture introduces coordination failure modes that are difficult to test exhaustively. Document these limitations clearly in your final report, not as an afterthought but as a central finding of your work. A capstone that honestly assesses what the system cannot do, and explains why, demonstrates deeper understanding than one that presents only successes. The known limitations listed below should be taken as a starting point; expand this list based on your own experience during development.

12.1 Critical Disclaimers

- **Not a clinical tool:** This is an educational prototype. It must never be used for actual clinical decision-making.
- **No real patient data:** All data is simulated. You must not use any real patient records, even de-identified ones, without explicit IRB approval.
- **No diagnostic claims:** Your system explicitly avoids making diagnoses. All outputs are framed as contextual summaries to support, not replace, clinical judgment.

12.2 Ethical Design Principles

- **Transparency:** Every output includes confidence scores and source citations. Your system explicitly flags what it does not know.

- **Human-in-the-loop:** Your system is designed as a decision-support tool. The clinician always makes the final decision.
- **Bias awareness:** You must document potential biases in your simulated data and prompt designs, and discuss how these could affect outputs in a real-world deployment.
- **Safety guardrails:** Critical alerts bypass synthesis and escalate immediately. Your system is designed to fail safe — when in doubt, it recommends human review rather than automated action.

12.3 Known Limitations

- LLM hallucination risk remains non-zero even with extensive guardrails; clinical deployment would require additional validation layers
- Simulated data cannot capture the full complexity and messiness of real clinical records
- The prototype does not address real-time latency requirements for acute care settings
- Multi-agent coordination introduces additional failure modes (agent disagreement, cascading errors) that are partially but not fully addressed
- The evaluation framework relies on your own gold standards, which may themselves contain errors or biases

13. References

A strong capstone project is grounded in the existing literature, drawing on published research to justify design decisions, contextualize the problem, and interpret results. This chapter provides a curated list of references organized by topic area that you should consult during the course of your project. The references span clinical informatics, remote patient monitoring, anamnesis and patient-reported data, LLM applications in healthcare, and prompt engineering and multi-agent systems. You are expected not merely to list these references but to engage with them substantively, citing specific findings that inform your design choices and evaluation criteria. The literature review you conduct in the first two weeks of the project timeline should produce a working familiarity with the key sources in each category.

The references provided here are intentionally drawn from peer-reviewed journals, major conference proceedings, and authoritative technical documentation to ensure that you are building on a foundation of credible evidence rather than blog posts and opinion pieces. In the healthcare domain, the quality of cited evidence matters enormously, because claims about clinical workflows, patient safety, and system effectiveness must be supported by rigorous research. Pay particular attention to the clinical informatics references, which document the specific data quality and interoperability challenges that motivate **ClinicalBridge**. The prompt engineering references provide both foundational techniques (chain-of-thought prompting, ReAct) and architectural patterns (generative agents, multi-agent systems) that directly inform your system design.

You are strongly encouraged to extend this reference list with additional sources you discover during your research, particularly recent publications that may have appeared after this document was written. The field of LLM applications in healthcare is evolving rapidly, and new findings about prompt engineering techniques, evaluation methodologies, and safety practices are published regularly. Any additional references should meet the same quality standards as those listed here: peer-reviewed or from authoritative technical sources, directly relevant to your project, and cited with sufficient specificity that a reader could locate and verify the claim. Your final project report should include a complete bibliography that demonstrates your engagement with the literature throughout the capstone period.

13.1 Clinical Context Gap and Health Informatics

- Wright, A., et al. (2015). Problem list completeness in electronic health records: A multi-site study and assessment of success factors. *International Journal of Medical Informatics*.
- Hravnak, M., et al. (2018). Cardiorespiratory instability before and after implementing an integrated monitoring system. *Critical Care Medicine*.
- Senbekov, M., et al. (2020). The recent progress and applications of digital technologies in healthcare: A review. *International Journal of Environmental Research and Public Health*.
- Adler-Milstein, J., & Jha, A. K. (2017). HITECH Act drove large gains in hospital electronic health record adoption. *Health Affairs*.

13.2 Remote Patient Monitoring

- Vegesna, A., et al. (2017). Remote patient monitoring via non-invasive digital technologies: A systematic review. *Telemedicine and e-Health*.
- Cvach, M. (2012). Monitor alarm fatigue: An integrative review. *Biomedical Instrumentation & Technology*.

- Noah, B., et al. (2018). Impact of remote patient monitoring on clinical outcomes: An updated meta-analysis of randomized controlled trials. NPJ Digital Medicine.

13.3 Anamnesis and Patient-Reported Data

- Bates, D. W., et al. (2014). Big data in health care: Using analytics to identify and manage high-risk and high-cost patients. Health Affairs.
- Bachmann, M., & Windak, A. (2020). Digital anamnesis in primary care: Potential and limitations. MDPI Healthcare.

13.4 LLM Applications in Healthcare

- Singhal, K., et al. (2023). Large language models encode clinical knowledge. Nature.
- Nori, H., et al. (2023). Capabilities of GPT-4 on medical competence examinations. arXiv preprint.
- Thirunavukarasu, A. J., et al. (2023). Large language models in medicine. Nature Medicine.

13.5 Prompt Engineering and Multi-Agent Systems

- Wei, J., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. NeurIPS.
- Yao, S., et al. (2023). ReAct: Synergizing reasoning and acting in language models. ICLR.
- Park, J. S., et al. (2023). Generative agents: Interactive simulacra of human behavior. UIST.
- LangChain Documentation. Retrieval-Augmented Generation and Agent Frameworks.